

El Viaje de los Datos: De lo Crudo a lo Refinado

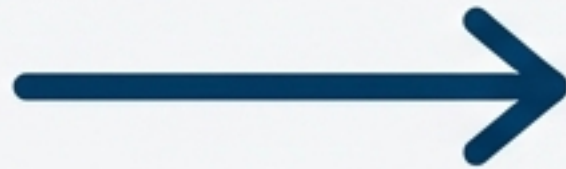
Técnicas Esenciales de Limpieza y Preparación con Pandas



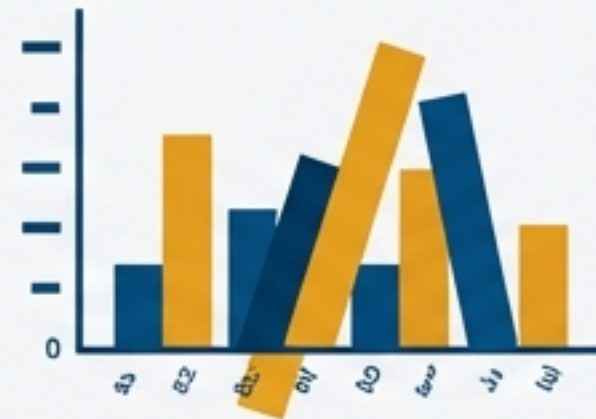
La Cruda Realidad: ¿Por Qué Invertir Tiempo en la Preparación?

Se introduce la idea de que los datos en su estado natural son imperfectos y no fiables. La preparación no es un paso previo, sino el fundamento de un análisis de calidad.

CAMINO 1: DATOS SUCIOS

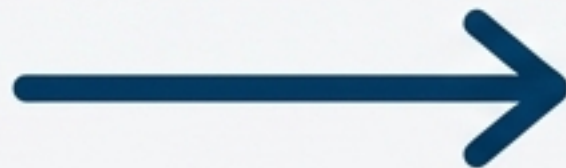


Resultado Incorrecto



Precisión en el Análisis: Datos sucios, como los incompletos o erróneos, distorsionan los resultados y llevan a conclusiones falsas.

CAMINO 2: DATOS LIMPIOS



Resultado Preciso



Eficiencia Operativa: Un set de datos bien preparado acelera el análisis y la modelización, permitiendo a los analistas centrarse en la extracción de insights.

Decisiones Informadas: La confianza en los datos es la base de las buenas decisiones. Un análisis preciso mejora la calidad de las decisiones estratégicas.

El Primer Obstáculo: Detección y Eliminación de Valores Faltantes

DataFrame Original y Código

prueba			
	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
2	NaN	NaN	NaN
3	4.0	8.0	12.0

```
# Detecta dónde están los nulos.  
prueba.isna()
```

```
# Elimina cualquier fila con un valor nulo.  
prueba.dropna()
```

```
# Solo elimina filas donde *todos* los valores son nulos.  
prueba.dropna(how="all")
```

Antes y Después

Después de `dropna()`

	A	B	C
0	1.0	5.0	9.0
3	4.0	8.0	12.0

Después de `dropna(how='all')`

	A	B	C
0	1.0	5.0	9.0
1	2.0	NaN	10.0
3	4.0	8.0	12.0

Explicación Estratégica

Eliminar es la opción más rápida y sencilla. Es ideal cuando los datos faltantes son escasos o cuando una fila o columna es mayormente inútil y no aporta información valiosa.

Rellenar los Huecos con Estrategia

DataFrame Original y Estrategias

original

	0	1	2
0	0.062692	NaN	NaN
1	-1.379858	NaN	NaN
2	1.934856	NaN	0.930063
3	-0.374948	NaN	0.557342
4	-0.532310	-1.088679	0.649480
5	0.781064	1.857998	0.300983
6	0.031965	-0.518349	-0.973399

Rellenar con un valor constante.

original.fillna(0)

Rellenar columnas específicas con valores diferentes.

original.fillna({1: 1.5, 2: 2.5})

Una técnica robusta: rellenar con la media.

original.fillna(original.mean())

Resultados

Resultados

Resultado de 'fillna(0)'

	0	1	2
0	0.062692	0.000000	0.000000
1	-1.379858	0.000000	0.000000
2	1.934856	0.000000	0.930063
3	-0.374948	0.000000	0.557342
4	-0.532310	-1.088679	0.649480
5	0.781064	1.857998	0.300983
6	0.031965	-0.518349	-0.973399

Resultado de 'fillna({...})'

	0	1	2
0	0.062692	1.500000	2.500000
1	-1.379858	1.500000	2.500000
2	1.934856	1.500000	0.930063
3	-0.374948	1.500000	0.557342
4	-0.532310	-1.088679	0.649480
5	0.781064	1.857998	0.300983
6	0.031965	-0.518349	-0.973399

Resultado de 'fillna(original.mean())'

	0	1	2
0	-0.251266	0.703885	-0.107652
1	1.873014	0.703885	-0.107652
2	1.663624	0.703885	-1.679340
3	2.073861	0.703885	-0.273092
4	-0.037209	0.049622	0.239458
5	-1.047683	0.978342	0.114876
6	1.412373	1.088691	1.059837

Explicación Estratégica

Rellenar preserva el tamaño de tu dataset. Usar la media o la mediana es una técnica común y robusta para no alterar significativamente la distribución estadística de los datos numéricos.

Asegurando la Unicidad: Eliminación de Duplicados

Detección y Eliminación

	A	B	C	D
0	1	X	10	100
1	2	Y	20	200
2	3	Z	30	300
3	1	X	10	100
4	2	Y	20	200
5	4	Z	30	300

```
# Devuelve una serie booleana.  
df.duplicated()
```

```
# Devuelve el DataFrame sin las filas duplicadas.  
df.drop_duplicates()
```



Resultados

```
0    False  
1    False  
2    False  
3     True  
4     True  
5    False  
dtype: bool
```

DataFrame Limpio

	A	B	C	D
0	1	X	10	100
1	2	Y	20	200
2	3	Z	30	300
5	4	Z	30	300

Justificación

Los duplicados inflan artificialmente las métricas como los conteos y pueden sesgar los modelos de Machine Learning. Su eliminación es un paso no negociable para la integridad del análisis.

Enriqueciendo los Datos Mediante Mapeo

Lógica de Negocio

	Salas	Especie	Total peces
0	1	Boquerón	10
1	2	Trucha	8
2	3	Barbo	15
3	4	Sardina	20
4	5	Carpa	9

```
pez_agua = {  
    'Boquerón': 'agua salada',  
    'Trucha': 'agua dulce',  
    'Barbo': 'agua dulce',  
    'Sardina': 'agua salada',  
    'Carpa': 'agua dulce'  
}
```

```
aquarium["Hábitat"] =  
aquarium["Especie"].map(get_habitat)
```

La función `map()` aplica el diccionario `pez_agua` a cada valor de la columna 'Especie', creando una nueva columna 'Hábitat'.

DataFrame Enriquecido

	Salas	Especie	Total peces	Hábitat
0	1	Boquerón	10	agua salada
1	2	Trucha	8	agua dulce
2	3	Barbo	15	agua dulce
3	4	Sardina	20	agua salada
4	5	Carpa	9	agua dulce

Beneficio

El método `map()` es una forma potente y altamente legible de aplicar una lógica de negocio o de categorización a tus datos. Es ideal para crear nuevas características (feature engineering) a partir de las existentes.

Corrigiendo y Estandarizando con `replace`

Valores Centinela

0	1
1	3
2	-999
3	0
4	2
5	-1000
6	-999
7	5
8	9

dtype: int64



Datos Estandarizados

0	1.0
1	3.0
2	NaN
3	0.0
4	2.0
5	NaN
6	NaN
7	5.0
8	9.0

dtype: float64

```
numeros.replace([-999, -1000], np.nan)
```

El Porqué

La función `replace` es ideal para limpiar datos categóricos con erratas o para corregir valores centinela. Este paso unifica la representación de los datos ausentes, preparándolos para un tratamiento consistente con `dropna` o `fillna`.

De Continuo a Categórico: Discretización de Datos

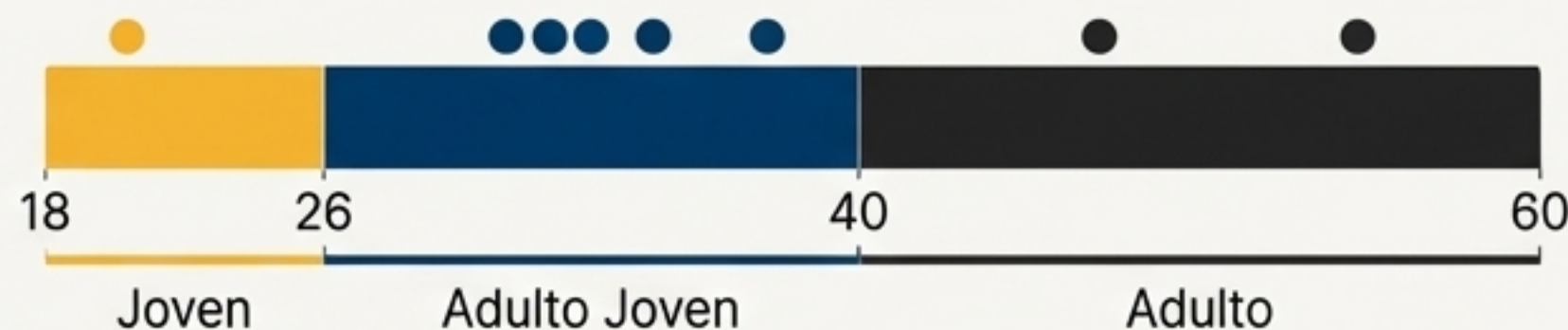
Conceptos y Código

cut(): Agrupa los datos en contenedores (bins) de tamaño fijo o definidos manualmente. Ideal para rangos de edad, niveles de ingresos, etc.

qcut(): Agrupa los datos en contenedores con el mismo número de elementos en cada uno (cuantiles). Útil para análisis de rankings o segmentación de clientes.

```
edad = [20, 49, 29, 28, 31, 26, 47, 32]
grupos = [18, 25, 40, 60]
division = pd.cut(edad, grupos,
                  labels=["Joven", "Adulto Joven", "Adulto"])
```

Resultados Visuales



Etiquetas Asignadas

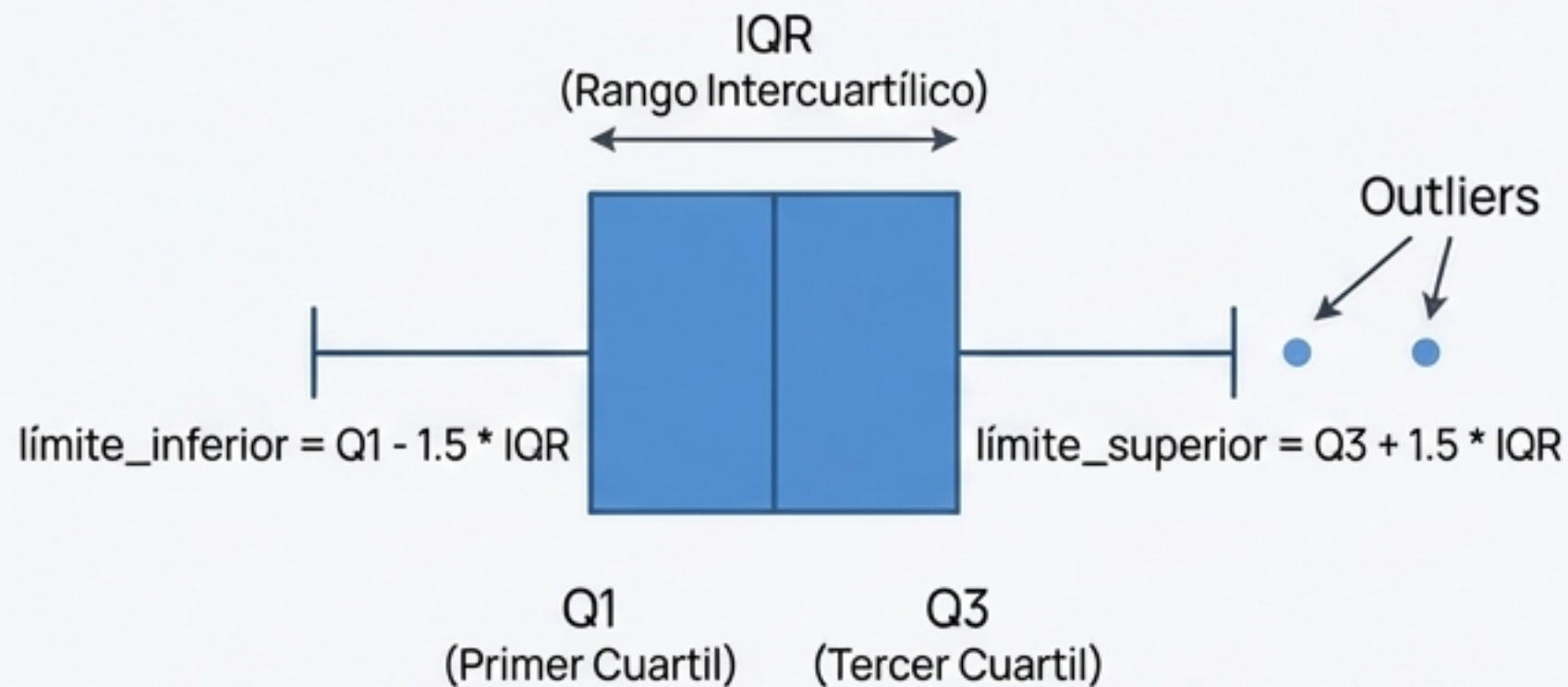
```
Categorical(['Joven', 'Adulto', 'Adulto Joven', 'Adulto Joven',
            'Adulto Joven', 'Adulto Joven', 'Adulto', 'Adulto Joven'])
Categories (3, object): ['Joven' < 'Adulto Joven' < 'Adulto']
```

Conteo por Grupo

Joven	1
Adulto Joven	5
Adulto	2

Domando los Extremos: Detección y Filtrado de Atípicos (Outliers)

La Lógica del IQR



Los outliers son datos que se alejan significativamente del resto. Pueden distorsionar métricas como la media y afectar a los modelos. El método del Rango Intercuartílico (IQR) es una técnica robusta para identificarlos.

Ejemplo Práctico

Filtrado de Ventas Anómalas

DataFrame Original

	Ventas
0	100
1	120
2	110
3	105
4	5000
5	130
6	140
7	9000
8	150
9	160

Atípicos Detectados

	Ventas
4	5000
7	9000

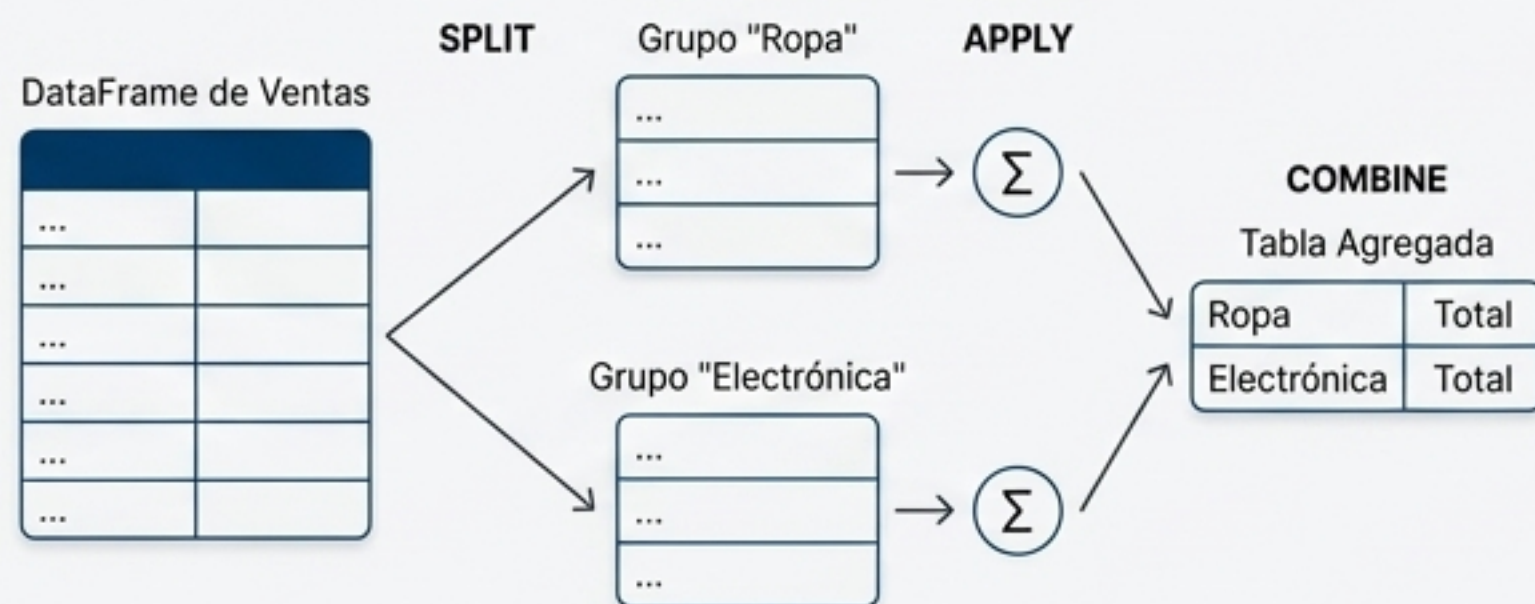
DataFrame Filtrado

	Ventas
0	100
1	120
2	110
3	105
5	130
6	140
8	150
9	160

La Piedra Angular del Análisis: Agrupar y Agregar con `groupby`

El Mecanismo

Split-Apply-Combine



```
# Agrupar por 'Categoria' y sumar las ventas
ventas_por_categoria = df.groupby('Categoria')['Ventas'].sum()
```

```
# También se puede usar mean(), count(), max() o varias a la vez:
df.groupby('...').agg(['sum', 'mean'])
```

De Datos a Métricas

Categoria	
Electrónica	450
Ropa	420

Name: Ventas, dtype: int64

El Poder

`groupby` es el motor para pasar de datos brutos a insights de negocio. Permite segmentar los datos en grupos significativos y analizar el comportamiento de cada uno de forma independiente, respondiendo a preguntas como '¿Cuál es nuestra categoría de producto más vendida?'.

Desbloqueando el Análisis Temporal con `datetime`

La Transformación

Antes: Solo Texto

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: object
```

```
df['Fecha_Venta'] = pd.to_datetime(df['Fecha_Venta'])
```



Después: Poder Temporal

```
0    2021-01-01
1    2021-02-01
2    2021-03-01
Name: Fecha_Venta, dtype: datetime64[ns]
```

Las Nuevas Capacidades

Convertir a `datetime` no es un cambio de formato, es un cambio de capacidad. Permite a pandas entender la estructura temporal de tus datos.

Ejemplo: Filtrar por Mes

```
# Filtrar todas las ventas realizadas en febrero.
df_filtrado = df[df['Fecha_Venta'].dt.month == 2]
```

Ahora puedes acceder a componentes como el año, mes o día de la semana (`.dt.year`, `.dt.month`, `.dt.dayofweek`), filtrar por rangos de fechas y realizar análisis de series temporales con facilidad.

Preparando para Machine Learning: Variables Dummy

El Problema

Los algoritmos de Machine Learning operan con números, no con texto. Necesitamos una forma de representar variables categóricas como 'Ciudad' de manera numérica.

La Solución: One-Hot Encoding

Se crea una nueva columna binaria (0 o 1) para cada categoría posible.

	Precio	Cantidad	Ciudad
0	5.95	1.98	Boston
1	3.31	4.73	Boston
2	8.14	7.14	Nueva York
3	8.43	6.64	Boston
4	6.90	0.17	Nueva York
5	8.29	1.38	Chicago
6	5.30	7.43	Chicago
7	0.82	2.67	Nueva York
8	6.11	0.90	Boston
9	6.42	5.83	Nueva York



Código

```
dummies = pd.get_dummies(df['Ciudad'])
```

Resultado

	Precio	Cantidad	Boston	Chicago	Nueva York
0	5.95	1.98	1	0	0
1	3.31	4.73	1	0	0
2	8.14	7.14	0	0	1
3	8.43	6.64	1	0	0
4	6.90	0.17	0	0	1
5	8.29	1.38	0	1	0
6	5.30	7.43	0	1	0
7	0.82	2.67	0	0	1
8	6.11	0.90	1	0	0
9	6.42	5.83	0	0	1

La función `get\_dummies` es el método estándar y esencial para preparar características categóricas antes de alimentar un modelo de regresión o clasificación.

El Arsenal Textual: Dominando la Limpieza Fina

Métodos de String Esenciales



`.lower()` / `.upper()`: Convertir a minúsculas/mayúsculas para estandarizar.



`.strip()`: Eliminar espacios en blanco al principio y al final.



`.replace('a', 'b')`: Sustituir subcadenas.



`.split(',')`: Dividir una cadena en una lista.

Búsqueda Avanzada con Regex

Para la limpieza y extracción de patrones complejos (emails, números de teléfono, etc.), las expresiones regulares son la herramienta definitiva.



`re.search()`: Encontrar la primera coincidencia de un patrón.

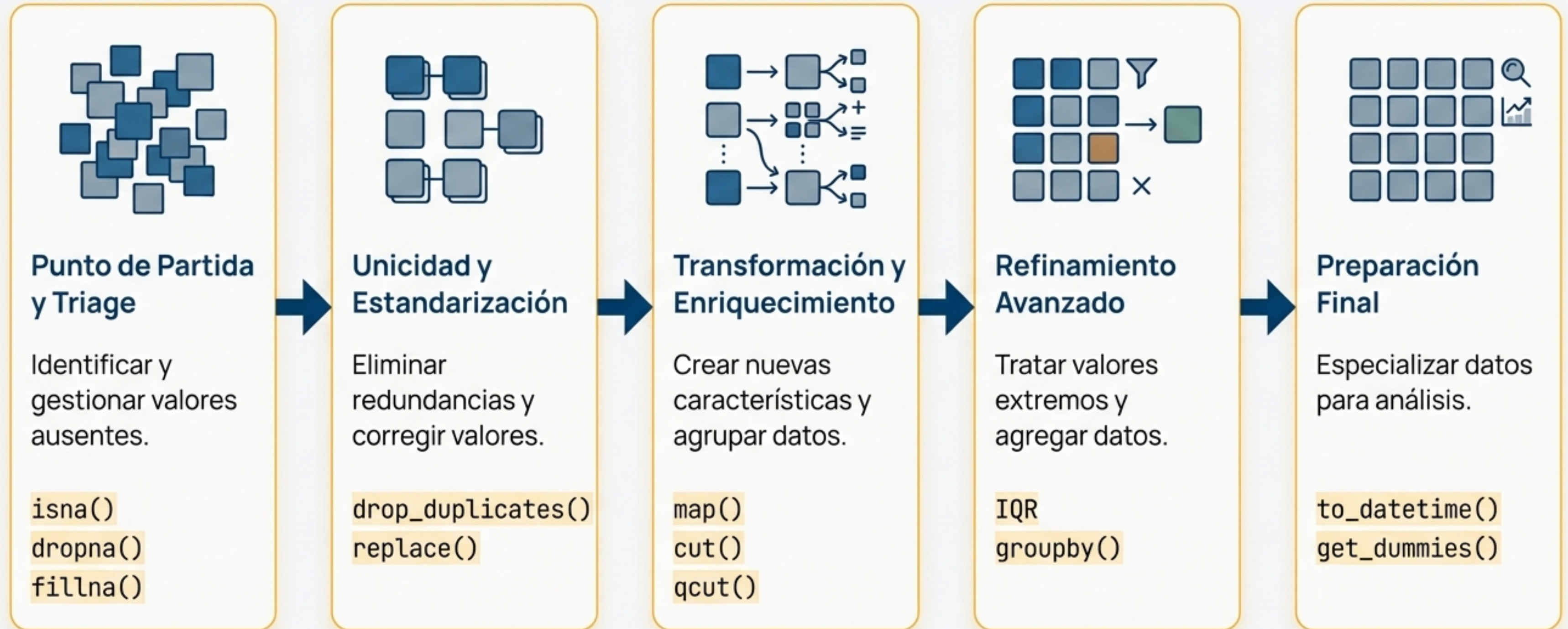


`re.findall()`: Encontrar todas las coincidencias.



`re.sub()`: Reemplazar patrones.

Resumen del Viaje: Nuestro Proceso de lo Crudo a lo Refinado



Los Datos Están Listos. El Análisis Comienza.



Una preparación de datos robusta no es un preámbulo; es el 90% de un análisis exitoso.
Ahora tienes las herramientas para construir una base sólida y fiable para tus insights.

Gracias.